

P3093R0

Attributes on expressions

Published Proposal, 2024-01-19

Author:

[Giuseppe D'Angelo](#)

Audience:

EWG, SG22

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Abstract

We propose to allow attributes on expressions.

Table of Contents

- 1 Changelog**
- 2 Motivation and Scope**
 - 2.1 Use cases
 - 2.2 Why doesn't C++ already support attributes on expressions?
- 3 Design Decisions**
 - 3.1 How to support expression attributes in the C++ grammar
 - 3.2 Rejected approaches
 - 3.2.1 Problems
- 4 Impact on the Standard**
- 5 Technical Specifications**

5.1 Proposed wording

6 Acknowledgements

References

Informative References

§ 1. Changelog

- R0
 - First submission. This paper has been split from [\[P2992R0\]](#).

§ 2. Motivation and Scope

The C++ grammar does not allow for attributes on arbitrary expressions.

For instance, a snippet like this:

```
int a = ([[attr]] f(1, 2, 3));
```

is ill-formed and rejected (with "interesting" error messages) by GCC, Clang and MSVC.

The following code is instead well-formed:

```
[[attr]] f(1, 2, 3);
```

The reason why this code is legal (and the previous is not) is that here the attribute appertains to the **statement**, not to the **expression**.

The grammar productions that are relevant are the *statement* and *expression-statement* productions ([\[stmt.pre\]](#), [\[stmt.expr\]](#)):

```
statement:  
    attribute-specifier-seqopt expression-statement
```

with

```
expression-statement:  
    expressionopt ;
```

and the production for *expression* itself ([\[expr.comma\]](#)):

```
expression:  
    assignment-expression  
    expression , assignment-expression
```

There are no other productions for expressions that allow for an attribute to be present, and this explains why the first code was illegal.

Here are some more examples of illegal placement of attributes on expressions:

```
// All currently ill-formed:  
  
// parenthesized version of the above:  
([[attr]] f(1, 2, 3));  
  
// attribute on function argument:  
process([[lock]] g()), 42);  
  
// in a comma expression:  
for (int i = 0; i < N; ++i, ([[discard]] f()))  
    doSomething(i);  
  
// in a member initialization list:  
struct S {  
    S(int i)  
        : m_i([[debug_only(check(i))]] i) {}  
  
    int m_i;  
};
```

This paper proposes to allow attributes on expressions.

§ 2.1. Use cases

This paper is a spin-off of [\[P2992R0\]](#), which is proposing the addition of the `[[discard(reason)]]` attribute as a more expressive version of a cast to `void`.

Such an attribute is meant to be used in places where a programmer deliberately wants to discard the result of a `[[nodiscard]]` function call, suppressing the warning that the implementation would otherwise raise.

As such, the attribute should be placed wherever a function call can appear, which is (in the general case) a sub-expression:

```
// returns an error code to be checked
[[nodiscard]] int f(int i);

// attribute on statement, already possible:
[[discard("f always succeeds for 42")]] f(42);

// attribute on expression, not currently possible:
for (int i = 0; i < N; ++i, ([[discard("f succeeds for inputs >= 0")]] f(i)
    doSomething(i);
```

One can concoct other similar situations: in [this blog post](#) Arthur O'Dwyer makes an example of using [\[P2946R1\]](#)'s `[[throws_nothing]]` attribute as a statement/expression attribute, as a way to make the compiler aware that a function to a non-`noexcept` function will in fact never throw an exception, and thus the compiler can do a better job at optimizing the call:

```
// as statement attribute:
[[throws_nothing]] f(42);

// as expression attribute:
struct S {
    S(int i)
        : m_i([[throws_nothing]] f(i)) {}

    int m_i;
};
```

§ 2.2. Why doesn't C++ already support attributes on expressions?

A possible reason for this is offered by [\[N2761\]](#) ("Towards support for attributes in C++"), where in Chapter 7 it is argued that a feature "used in expressions as opposed to declarations" should "use/reuse a keyword" instead.

Adding a keyword has however a very high barrier and cost for the language and ecosystem.

A keyword is also fundamentally different from an attribute: a keyword is not ignorable, while an attribute can be ignored. A vendor cannot add vendor-specific keywords without forking the language, but they can add vendor-specific attributes. With the current rules on attribute ignorability (cf. [\[P2552R3\]](#)), standard attributes have "optional semantics", while any other attribute is either picked up by the implementation or it must be ignored ([\[dcl.attr.grammar\]/6](#)).

For this reason we think that attributes should be supported on expressions.

§ 3. Design Decisions

§ 3.1. How to support expression attributes in the C++ grammar

An "obvious" modification of the *expression* production to introduce attributes could look like this:

```
expression:  
    attribute-specifier-seqopt assignment-expression // not proposed!  
    expression , assignment-expression
```

This change however clashes with the *statement* production:

```
statement:  
    attribute-specifier-seqopt expression-statement  
  
expression-statement:  
    expressionopt ;
```

resulting in an ambiguity for a statement like this:

```
[[attribute]] x = 42; // is this a statement attribute or an expression att
```

Changing the meaning of the snippet above would be a source-incompatible break, because it could alter the semantics of the attribute and/or make the code ill-formed (in case the attribute can only appertain to statements). This is something that we do not want to do.

We also do not want to complicate the grammar and/or the semantics of attributes, for instance by:

- having each attribute "state" somehow if it should apply to statements or expressions;
- adding normative wording to disambiguate the above case in favor of the status quo, that is, make the attribute always appertain to the statement. This would still leave us with the problem of how to apply an attribute to the expression in the snippet.



Instead, we are going to propose a different change in the grammar: allow attributes only on **parenthesized expressions**. In this case there's a token (the open parenthesis) that separates the expression from anything preceding it, avoiding the clash.

The extra verbosity of having to use parenthesis is justified by the fact that attributes are rarely used anyhow.

This is the grammar change that we are proposing:

```
primary-expression:
  literal
  this
  ( attribute-specifier-seqopt expression )
  id-expression
  lambda-expression
  fold-expression
  requires-expression
```

We also also going to special-case the semantics of parenthesized expressions, so that their attribute applies to the inner expression.

Here's some examples of attributes on expressions that this approach allows for:

```
int a[10];

[[attr]] a[0] = x + y;    // attr applies to the statement
([[attr]] a[1]) = x + y; // attr applies to `a[1]`
```

```

a[2] = [[attr]] x + y;      // ill-formed
a[3] = ([[attr]] x) + y;    // attr applies to `x`
a[4] = ([[attr]] x + y);    // attr applies to `x + y`
a[4] = ([[attr]] (x + y));  // ditto, parenthesized sub-expression
[[attr]] a[6] = x + y;      // attr applies to `a[6] = x + y`

// attr1 applies to the whole requires-expression
// attr2 applies to `c.foo()`
// attr3 applies to `*c`
template <typename T>
concept C =
    ([[attr1]] requires (C c)
    {
        ([[attr2]] c.foo());
        { ([[attr3]] *c) } -> convertible_to<bool>;
    });

// attr1 applies to the statement
// attr2 applies to the overall expression
// attr3 applies to the closure's function call operator
// attr4 applies to the closure's function call operator's type
[[attr1]] ( [[attr2]] [] [[attr3]] () [[attr4]] {} () );

```

The previous examples would all become well-formed:

```

// OK, applies to the entire expression
[[attr]] f(1, 2, 3));

// OK, applies to `g()`
process([[lock]] g()), 42);

// OK, applies to `f()`
for (int i = 0; i < N; ++i, ([[discard]] f()))
    doSomething(i);

// OK, applies to `i`
struct S {
    S(int i)
        : m_i([[debug_only(check(i))]] i) {}

```

```
int m_i;
};
```

Despite the extra verbosity, we strongly believe that by using parenthesis, it becomes very clear to which sub-expression an attribute appertains to.

We are also confident that this grammar change does not result in any ambiguity or conflicts. (If it did, such conflicts would already exist with the grammar for statements.)

§ 3.2. Rejected approaches

Given the grammar clash described above, if we do not want users to have to add parenthesis to every expression they want to tag with an attribute, we could decide to allow attributes on the **right hand side** of an *expression*.

We could modify the *expression* production as follows:

```
expression:
  assignment-expression attribute-specifier-seqopt // not proposed!
  expression , assignment-expression
```

Here are some examples of what this approach would look like:

```
int a[10];

a[1] = x + y [[attr]]; // attr applies to `a[1] = x + y`
a[2] = x + (y [[attr]]); // attr applies to `y`
a[3] = ((x+y) [[attr]]); // attr applies to `x+y`
a[4] = (x+y [[attr]]); // attr applies to `x+y`

// Attributes can only be applied on expressions, and not (unparenthesized)
// assignment-expressions, primary-expressions, etc.:
a[5] = x [[attr]] + y; // ill-formed
a[i [[attr]] ] = 42; // ill-formed
a[6] [[attr]] = 123; // ill-formed

x [[attr]] = -1; // ill-formed

int x = [[attr]] f(); // ill-formed
```

```

int y = f() [[attr]];    // ill-formed (the initializer wants an assignmer
int z = (f() [[attr]]);  // OK: attr applies to `f()`

// We can apply attributes to arbitrary sub-expressions by parenthesizing t
// attr1 applies to `x`
// attr2 applies to `y+2`
// attr3 applies to the whole expression
(x [[attr1]]) = (y+2 [[attr2]]) [[attr3]];

// attr1 applies to `c.foo()`
// attr2 applies to `*c`
// attr3 applies to the whole requires-expression
template <typename T>
concept C = (requires (C c) {
    c.foo() [[attr1]];
    { (*c) [[attr2]] } -> convertible_to<bool>;
} [[attr3]]);

// attr1 applies to the statement
// attr2 applies to the closure's function call operator
// attr3 applies to the closure's function call operator's type
// attr4 applies to the overall expression
[[attr1]] [] [[attr2]] () [[attr3]] {} () [[attr4]];

// attr applies to the closure's function call operator, and not
// to the requires-expression in the requires-clause, as per
// [expr.prim.lambda.general]/3
[]<typename T> requires
    requires (T t) { *t; }
    [[attr]] () {};

```

§ 3.2.1. Problems

This approach has a number of shortcomings.

The biggest one is purely esthetical: having attributes on the right hand side of the entity they appertain to feels very unnatural, an impedance mismatch with the rest of the language. In this snippet:

```
result = x + y [[attr]];
```

it's not obvious at all that the attribute is being applied to the entire expression (and not just to `y` or to `x + y`).

A second limitation is due to the fact that, by changing only the *expression* grammar production, we would not actually allow attributes on **all** possible kinds of sub-expressions. For instance, this would be ill-formed:

```
result = x [[attr]] + y; // still illegal with the grammar change
```

because `x` isn't a result of the *expression* production.

Complicating the grammar to allow for attributes "everywhere" is likely not worth the effort, because one can always wrap a subexpression in parenthesis in order to apply an attribute to it. Still, the above code could be surprising.

Finally, this approach also conflicts with some existing grammar productions. We are aware of at least two.

1. The production(s) for new expressions for arrays, added by [\[N3033\]](#) as resolution of [\[CWG951\]](#). In [\[expr.new\]](#) there are the following productions:

```
noptr-new-declarator:  
    [ expressionopt ] attribute-specifier-seqopt  
    noptr-new-declarator [ constant-expression ] attribute-specifier-seq
```

with the attribute appertaining to the associated array type. This means that `auto ptr = (new T[123] [[someattribute]]);` is legitimate code today.

We are unsure about a use case for allowing attributes specifically on new expressions for arrays. (Rather than applying an attribute on the array type right into the new expression, can't the same intent be better expressed by having an attribute on e.g. a type alias to the array type, while allowing the attribute in `new` to appertain to the expression?)

2. The production(s) for conversion functions in [\[class.conv.fct\]](#), added by [\[N2761\]](#). A *primary-expression* can contain a *conversion-function-id* as subexpression, and the associated grammar allows attributes at the end:

```
ptr-declarator ( parameter-declaration-clause ) cv-qualifier-seqopt  
ref-qualifier-seqopt noexcept-specifieropt attribute-specifier-seqopt
```

Here the attribute appertains to the function type ([\[dcl.fct\]/1](#)). For instance, this code is legitimate:

```
struct S { operator int() const; };  
auto ptr = (&S::operator int [[attribute]]);
```

A similar example is available in [\[P2173R1\]](#).

An implementation-specific attribute can, in principle, be used to select a specific overload (since they apply to the type):

```
// example and explanation courtesy of Richard Smith  
struct S {  
    operator int() [[vendor::attr1]] const; // #1  
    operator int() [[vendor::attr2]] const; // #2  
};  
  
auto ptr = (&S::operator int [[vendor::attr2]]); // select #2
```

How to solve these cases? A possible solution could be to simply enshrine that, in case of an ambiguity, the tie is resolved in favour of the status-quo. If instead grammar changes for these productions are wanted, unfortunately we are unable to evaluate the real-world breakage that could result.

We do not feel comfortable at introducing breaking changes, so, once more, we are not pursuing this approach.

§ 4. Impact on the Standard

This proposal is a core language extension. It proposes changes to the C++ grammar to allow attributes on expressions.

No changes are required in the Standard Library.

§ 5. Technical Specifications

All the proposed changes are relative to [\[N4971\]](#).

§ 5.1. Proposed wording

Modify the grammar productions for *primary-expression* in [\[expr.prim\]](#) and in [\[gram.expr\]](#) as shown:

```
primary-expression:  
    literal  
    this  
    ( attribute-specifier-seqopt expression )  
    id-expression  
    lambda-expression  
    fold-expression  
    requires-expression
```

In [\[expr.prim.paren\]](#), append a new paragraph:

2. The optional *attribute-specifier-seq* appertains to the expression, unless the expression is itself a parenthesized expression, in which case it appertains to the expression between the parentheses.



Modify [\[dcl.attr.grammar\]/5](#) as shown:

5. Each *attribute-specifier-seq* is said to *appertain* to some entity ~~or~~, statement or expression, identified by the syntactic context where it appears ([stmt.stmt], [dcl.dcl], [dcl.decl], [\[expr.prim\]](#)). If an *attribute-specifier-seq* that appertains to some entity ~~or~~, statement or expression contains an attribute or alignment-specifier that is not allowed to apply to that entity ~~or~~, statement or expression, the program is ill-formed. If an *attribute-specifier-seq* appertains to a friend declaration ([class.friend]), that declaration shall be a definition.



Modify the "Feature-test macros" table in [\[tab:cpp.predefined.ft\]](#), by adding a new row as shown:

Macro name	Value
<code>__cpp_expression_attributes</code>	<code>YYYYMML</code>

with `YYYYMML` determined as usual.

§ 6. Acknowledgements

Thanks to KDAB for supporting this work.

All remaining errors are ours and ours only.

§ References

§ Informative References

[CWG951]

Sean Hunt. *Problems with attribute-specifiers*. 5 August 2009. CD2. URL: <https://wg21.link/cwg951>

[GUARDNOUNORVERB]

Arthur O'Dwyer. *Guard nouns or guard verbs?*. URL: <https://quuxplusone.github.io/blog/2023/11/11/guard-nouns-or-guard-verbs/>

[N2761]

J. Maurer, M. Wong. *Towards support for attributes in C++ (Revision 6)*. 18 September 2008. URL: <https://wg21.link/n2761>

[N3033]

Daveed Vandevorode. *Core issue 951: Various Attribute Issues*. 5 February 2010. URL: <https://wg21.link/n3033>

[N4971]

Thomas Köppe. *Working Draft, Programming Languages — C++*. 18 December 2023. URL: <https://wg21.link/n4971>

[P2173R1]

Daveed Vandevorode, Inbal Levi, Ville Voutilainen. *Attributes on Lambda-Expressions*. 9 December 2021. URL: <https://wg21.link/p2173r1>

[P2552R3]

Timur Doumler. *[On the ignorability of standard attributes](https://wg21.link/p2552r3)*. 14 June 2023. URL:
<https://wg21.link/p2552r3>

[P2946R1]

Pablo Halpern. *[A flexible solution to the problems of `noexcept`](https://wg21.link/p2946r1)*. 16 January 2024. URL:
<https://wg21.link/p2946r1>

[P2992R0]

Giuseppe D'Angelo. *[Attribute `\[\[discard\]\]` and attributes on expressions](https://wg21.link/p2992r0)*. 10 October 2023. URL:
<https://wg21.link/p2992r0>